



Contenido

UNIDAD 8 Bloque I. Desarrollo de aplicaciones en Laravel.....	2
Instalación de Laravel con Xampp	2
Estructura	3
Funcionamiento básico	4
Rutas	5
Rutas básicas	5
Añadir parámetros a las rutas	6
Artisan.....	8
Vistas	9
Plantillas mediante <i>Blade</i>	10



UNIDAD 8 Bloque I. Desarrollo de aplicaciones en Laravel

Bloque I. Introducción a Laravel

LARAVEL es un framework de código abierto para desarrollar aplicaciones y servicios web con PHP . Su filosofía es desarrollar código PHP de forma elegante y simple. Fue creado en 2011 y tiene una gran influencia de frameworks como Ruby on Rails, Sinatra y ASP.NET MVC.

Instalación de Laravel con Xampp

Debemos de tener instalado:

- Un servidor de aplicaciones web. En este caso: XAMPP
- Composer para descargar y gestionar las dependencias del Framework. Se trata de un administrador de dependencias para PHP que nos permite descargar paquetes desde un repositorio para agregarlo a nuestro proyecto. Por defecto, se agregan a una carpeta llamada /vendor. De esta manera evitamos hacer las búsquedas manualmente y el mismo Composer se puede encargar de actualizar las dependencias que hayamos descargado por una nueva versión. Para instalar Composer en Windows debemos descargarlo de su [página oficial](#) y en la sección Windows Installer, haz click en Composer- Setup.exe.

Composer es un gestor de dependencias en proyectos, para programación en PHP. Eso quiere decir que nos permite gestionar (declarar, descargar y mantener actualizados) los paquetes de software en los que se basa nuestro proyecto PHP. Se ha convertido en una herramienta de cabecera para cualquier desarrollador en este lenguaje que aprecie su tiempo y el desarrollo ágil.

Para instalar Laravel los requerimientos son versión de PHP >5.4. Usando la consola/terminal vamos a la carpeta C:/xampp/htdocs o subcarpeta dentro de la misma y allí escribimos

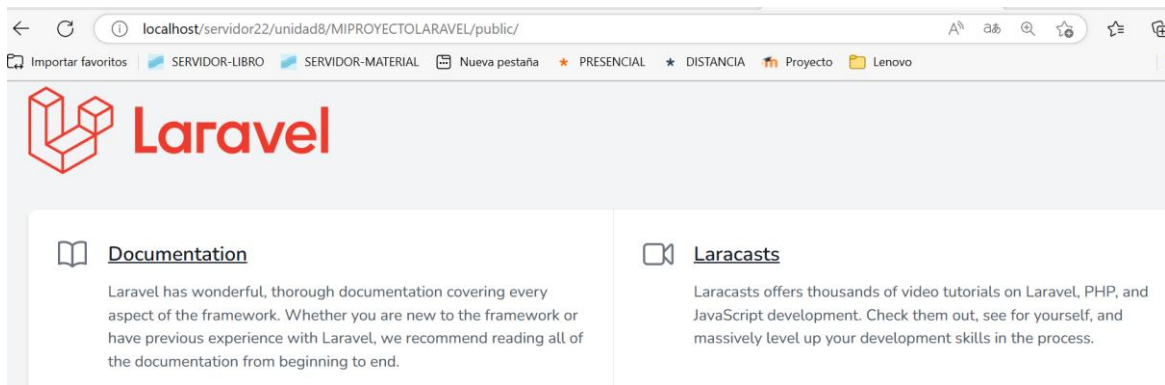
```
composer create-project --prefer-dist laravel/laravel MIPROYECTOLARAVEL
```

donde “MIPROYECTOLARAVEL” es el nombre de mi proyecto. Ahora Composer empezará a descargar e instalar el Framework y todas sus dependencias.



```
C:\xampp\htdocs\SERVIDOR22\UNIDAD8>composer create-project --prefer-dist laravel/laravel MIPROYECTOLARAVEL
Creating a "laravel/laravel" project at "./MIPROYECTOLARAVEL"
Info from https://repo.packagist.org: #StandWithUkraine
Installing laravel/laravel (v9.5.0)
- Installing laravel/laravel (v9.5.0): Extracting archive
Created project in C:\xampp\htdocs\SERVIDOR22\UNIDAD8\MIPROYECTOLARAVEL
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 107 installs, 0 updates, 0 removals
- Locking brick/math (0.10.2)
- Locking dflydev/dot-access-data (v3.0.2)
- Locking doctrine/deprecations (v1.0.0)
- Locking doctrine/inflector (2.0.6)
- Locking doctrine/instantiator (2.0.0)
- Locking doctrine/lexer (2.1.0)
- Locking dragonmantank/cron-expression (v3.3.2)
- Locking egulias/email-validator (3.2.5)
```

Ahora ya tendremos Laravel instalado y listo para empezar a programar con él.



Estructura

Primero que nada vamos a ver la estructura de nuestro proyecto, para así entender que hay dentro de las principales carpetas.

Windows-SSD (C:) > xampp > htdocs > SERVIDOR22 > UNIDAD8 > MIPROYECTOLARAVEL

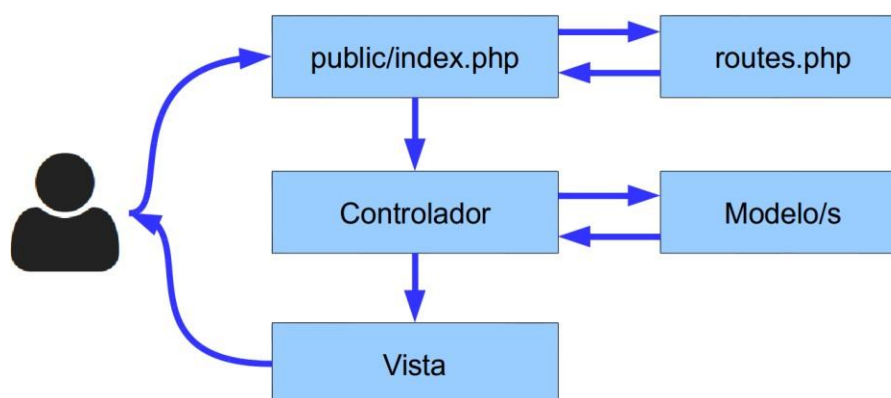
Nombre	Fecha de modificación	Tipo	Tamaño
app	02/01/2023 15:45	Carpeta de archivos	
bootstrap	02/01/2023 15:45	Carpeta de archivos	
config	02/01/2023 15:45	Carpeta de archivos	
database	02/01/2023 15:45	Carpeta de archivos	
lang	02/01/2023 15:45	Carpeta de archivos	
public	02/01/2023 15:45	Carpeta de archivos	
resources	02/01/2023 15:45	Carpeta de archivos	
routes	02/01/2023 15:45	Carpeta de archivos	
storage	02/01/2023 15:45	Carpeta de archivos	
tests	02/01/2023 15:45	Carpeta de archivos	
vendor	16/01/2023 10:05	Carpeta de archivos	
.editorconfig	02/01/2023 15:45	Archivo EDITORCON...	1 KB
.env	16/01/2023 10:05	Archivo ENV	2 KB
.env.example	02/01/2023 15:45	Archivo EXAMPLE	2 KB
.gitattributes	02/01/2023 15:45	Archivo GITATTRIBUT...	1 KB
.gitignore	02/01/2023 15:45	Archivo GITIGNORE	1 KB



- /app - Contiene los controladores, modelos, vistas y configuraciones de la aplicación. En esta carpeta escribiremos la mayoría del código para que nuestra aplicación funcione. En la raíz tenemos un solo modelo: "user.php" que se crea por defecto
- /app/Http.-
- Controllers son los que interaccionan con los modelos
- Middleware son filtros de seguridad cuando se envía una ruta, un formulario...
- bootstrap- son archivos del sistema
- /config.- todos los archivos de configuración del sistema
- App.php tenemos los namespace para acceder a las librerías internas de Laravel, si descargamos algo nuevo para nuestra aplicación hay que instalar los namespace aquí.
- Database.php se configura la B.D. que ya contiene mysql.
- Filesystems.php maneja discos internos en laravel: imágenes, videos,...
- /database/Migrations: se crea la estructura para la B.D., tablas....
- /public - Es la única carpeta a la que los usuarios de la aplicación pueden acceder. Todo las peticiones y solicitudes a la aplicación pasan por esta carpeta, ya que en ella se encuentra el index.php, este archivo es el que inicia todo el proceso de ejecución del framework. En este directorio también se alojan los archivos CSS, Javascript, imágenes y otros archivos que se quieran hacer públicos.
- /resources
- Lang : carpeta para los lenguajes
- Views: las vistas
- Vendor
- Welcome.blade.php: que es la página de inicio
- /routes.- las rutas
- Web.php es la más importante, aquí se definen las rutas para interpretar las solicitudes que el usuario hace al sistema
- /storage.- discos internos de laravel
- /vendor - En esta carpeta se alojan todas las librerías que conforman el framework y sus dependencias.
- /lang - En esta carpeta se guardan archivos PHP que contienen Arrays con los textos de diferentes lenguajes, en caso que se desee que la aplicación se pueda traducir.
- /app/model
- /app/views - Este directorio contiene las plantillas de HTML que usan los controladores para mostrar la información. Hay que tener en cuenta que en esta carpeta no van los Javascript, CSS o imágenes, ese tipo de archivos van alojados en la carpeta /public.

Funcionamiento básico

El funcionamiento básico que sigue Laravel tras una petición web a una URL de nuestro sitio es el siguiente:



Todas las peticiones entran a través del fichero `public/index.php`, el cual en primer lugar comprobará en el fichero de rutas (`routes/web.php`) si la URL es válida y en caso de serlo a que controlador tiene que hacer la petición.

A continuación se llamará al método del controlador asignado para dicha ruta. Como hemos visto, el controlador es el punto de entrada de las peticiones del usuario, el cual, dependiendo de la petición:

Accederá a la base de datos (si fuese necesario) a través de los "modelos" para obtener datos (o para añadir, modificar o eliminar).

Tras obtener los datos necesarios los preparará para pasárselos a la vista.

En el tercer paso el controlador llamará a una vista con una serie de datos asociados, la cual se preparará para mostrarse correctamente apartir de los datos de entrada y por último se mostrará al usuario.

Rutas

Las rutas de nuestra aplicación se tienen que definir en el fichero `routes/web.php`. Este es el punto centralizado para la definición de rutas y cualquier ruta no definida en este fichero no será válida, generando una excepción (lo que devolverá un error 404).

Las rutas, en su forma más sencilla, pueden devolver directamente un valor desde el propio fichero de rutas, pero también podrán generar la llamada a una vista o a un controlador.

Rutas básicas

Las rutas, además de definir la URL de la petición, también indican el método con el cual se ha de hacer dicha petición. Los dos métodos más utilizados y que empezaremos viendo son las peticiones tipo GET y tipo POST. Por ejemplo, para definir una petición tipo GET

tendríamos que añadir el siguiente código a nuestro fichero web.php:

```
Route::get('/', function()
{
return '¡Hola mundo!';
});
```

Este código se lanzaría cuando se realice una petición tipo GET a la ruta raíz de nuestra aplicación. Si estamos trabajando en local esta ruta sería `http://localhost` pero cuando la web esté en producción se referiría al dominio principal, por ejemplo: `http://www.dirección-de-tu-web.com`. Es importante indicar que si se realiza una petición tipo POST o de otro tipo que no sea GET a dicha dirección se devolvería un error ya que esa ruta no está definida.

Para definir una ruta tipo POST se realizaría de la misma forma pero cambiando el verbo GET por POST:

```
Route::post('foo/bar', function()
{
return '¡Hola mundo!';
});
```

En este caso la ruta apuntaría a la dirección URL `foo/bar` (`http://localhost/foo/bar`) o (`http://www.dirección-de-tu-web.com/foo/bar`).

Si queremos que una ruta se defina a la vez para get y post lo podemos hacer añadiendo un array con los tipos, de la siguiente forma:

```
Route::match(array('GET', 'POST'), '/', function()
{
return '¡Hola mundo!';
});
```

O para cualquier tipo de petición HTTP utilizando el método any:

```
Route::any('foo', function()
{
return '¡Hola mundo!';
});
```

Añadir parámetros a las rutas

Si queremos añadir parámetros a una ruta simplemente los tenemos que indicar entre llaves `{}` a continuación de la ruta, de la forma:

```
Route::get('user/{id}', function($id)
{
return 'User '.$id;
});
```

En este caso estamos definiendo la ruta `/user/{id}`, donde `id` es requerido y puede ser

cualquier valor. En caso de no especificar ningún id se produciría un error. El parámetro se le pasará a la función, el cual se podrá utilizar (como veremos más adelante) para por ejemplo obtener datos de la base de datos, almacenar valores, etc.

También podemos indicar que un parámetro es opcional simplemente añadiendo el símbolo ? al final (y en este caso no daría error si no se realiza la petición con dicho parámetro):

```
Route::get('user/{name?}', function($name = null)
{
    return $name;
});

// También podemos poner algún valor por defecto...

Route::get('user/{name?}', function($name = 'Javi')
{
    return $name;
});
```

Laravel también permite el uso de expresiones regulares para validar los parámetros que se le pasan a una ruta. Por ejemplo, para validar que un parámetro esté formado solo por letras o solo por números:

```
Route::get('user/{name}', function($name)
{
    //
})
->where('name', '[A-Za-z]+');

Route::get('user/{id}', function($id)
{
    //
})
->where('id', '[0-9]+');

// Si hay varios parámetros podemos validarlos usando un array:
Route::get('user/{id}/{name}', function($id, $name)
{
    //
})
->where(array('id' => '[0-9]+', 'name' => '[A-Za-z]+'))
```

A estas alturas ya tendríamos que ser capaces de crear una web con contenido estático, simplemente modificando el fichero de rutas y devolviendo todo el contenido desde aquí. Pero esto no es lo correcto porque terminaríamos con un fichero web.php inmenso con todo el código mezclado en el mismo archivo. En las siguientes secciones vamos a ver como separar el código de las vistas y más adelante añadiremos los controladores.



Artisan

Laravel incluye un interfaz de línea de comandos (CLI, *Command line interface*) llamado **Artisan**. Esta utilidad nos va a permitir realizar múltiples tareas necesarias durante el proceso de desarrollo o despliegue a producción de una aplicación, por lo que nos facilitará y acelerará el trabajo.

Entre las herramientas que Laravel nos proporciona para el desarrollo de aplicaciones se encuentra Artisan, la interfaz de línea de comandos (CLI por sus siglas en inglés de Command-line interface), la cual es un medio para la interacción con la aplicación donde los usuarios (en este caso los desarrolladores) dan instrucciones en forma de línea de texto simple o línea de comando. Artisan está basado en el componente Console de Symfony y nos ofrece un conjunto de comandos que nos pueden ayudar a realizar diferentes tareas durante el desarrollo e incluso cuando la aplicación se encuentra en producción.

Para ver una lista de todas las opciones que incluye Artisan podemos ejecutar el siguiente comando en una consola o terminal del sistema en la carpeta raíz de nuestro proyecto:

```
php artisan list  
# O simplemente:  
php artisan
```

Si queremos obtener una ayuda más detallada sobre alguna de las opciones de Artisan simplemente tenemos que escribir la palabra *help* delante del comando en cuestión, por ejemplo:

```
php artisan help migrate
```

Para ver un listado con todas las rutas que hemos definido en el fichero `routes.php` podemos ejecutar el comando:

```
php artisan route:list
```

Esto nos mostrará una tabla con el método, la dirección, la acción y los filtros definidos para todas las rutas. De esta forma podemos comprobar todas las rutas de nuestra aplicación y asegurarnos de que esté todo correcto.

A través de la opción de Artisan `make` podemos generar diferentes componentes de Laravel (controladores, modelos, filtros, etc.) como si fueran plantillas, esto nos ahorrará mucho trabajo y podremos empezar a escribir directamente el contenido del componente. Por ejemplo, para crear un nuevo controlador tendríamos que escribir:

```
php artisan make:controller TaskController
```



Vistas

Las vistas son la forma de presentar el resultado (una pantalla de nuestro sitio web) de forma visual al usuario, el cual podrá interactuar con él y volver a realizar una petición. Las vistas además nos permiten separar toda la parte de presentación de resultados de la lógica (controladores) y de la base de datos (modelos). Por lo tanto no tendrán que realizar ningún tipo de consulta ni procesamiento de datos, simplemente recibirán datos y los prepararán para mostrarlos como HTML.

Las vistas se almacenan en la carpeta `resources/views` como ficheros PHP, y por lo tanto tendrán la extensión `.php`. Contendrán el código HTML de nuestro sitio web, mezclado con los assets (CSS, imágenes, Javascripts, etc. que estarán almacenados en la carpeta `public`) y algo de código PHP (o código Blade de plantillas, como veremos más adelante) para presentar los datos de entrada como un resultado HTML.

A continuación se incluye un ejemplo de una vista simple, almacenada en el fichero `resources/views/home.php`, que simplemente mostrará por pantalla ¡Hola <nombre>!, donde <nombre> es una variable de PHP que la vista tiene que recibir como entrada para poder mostrarla.

```
<html>
<head>
<title>Mi Web</title>
</head>
<body>
<h1>Hola <?php echo $nombre; ?>!</h1>
</body>
</html>
```

Una vez tenemos una vista tenemos que asociarla a una ruta para poder mostrarla. Para esto tenemos que ir al fichero `routes.php` como hemos visto antes y escribir el siguiente código:

```
Route::get('/', function()
{
return view('home', array('nombre' => 'Pedro'));
});
```

En este caso estamos definiendo que la vista se devuelva cuando se haga una petición tipo GET a la raíz de nuestro sitio. El único cambio que hemos hecho con respecto a lo que vimos en la sección anterior de rutas ha sido en el valor devuelto por la función, el cual genera la vista usando el método `view` y la devuelve. Esta función recibe como parámetros:

- El nombre de la vista (en este caso `home`), el cual será un fichero almacenado en la carpeta `views`, acordaros que la vista anterior de ejemplo la habíamos guardado en `resources/views/home.php`. Para indicar el nombre de la vista se utiliza el

mismo nombre del fichero pero sin la extensión .php.

- Como segundo parámetro recibe un array de datos que se le pasarán a la vista. En este caso la vista recibirá una variable llamada \$nombre con valor "Pedro".

Como hemos visto para referenciar una vista únicamente tenemos que escribir el nombre del fichero que la contiene pero sin la extensión .php. En el ejemplo, para cargar la vista almacenada en el fichero home.php la referenciamos mediante el nombre home, sin la extensión .php ni la ruta resources/views.

Las vistas se pueden organizar en sub-carpetas dentro de la carpeta resources/views, por ejemplo podríamos tener una carpeta resources/views/user y dentro de esta todas las vistas relacionadas, como por ejemplo login.php, register.php o profile.php. En este caso para referenciar las vistas que están dentro de sub-carpetas tenemos que utilizar la notación tipo "dot", en la que las barras que separan las carpetas se sustituyen por puntos. Por ejemplo, para referenciar la vista resources/views/user/login.php usaríamos el nombre user.login, o la vista resources/views/user/register.php la cargaríamos de la forma:

```
Route::get('register', function()
{
    return view('user.register');
});
```

Como hemos visto, para pasar datos a una vista tenemos que utilizar el segundo parámetro del método view, el cual acepta un array asociativo. En este array podemos añadir todas las variables que queramos utilizar dentro de la vista, ya sean de tipo variable normal (cadena, entero, etc.) u otro array o objeto con más datos. Por ejemplo, para enviar a la vista profile todos los datos del usuario cuyo id recibimos a través de la ruta tendríamos que hacer:

```
Route::get('user/profile/{id}', function($id)
{
    $user = // Cargar los datos del usuario a partir de $id
    return view('user.profile', array('user' => $user));
});
```

Plantillas mediante *Blade*

Laravel utiliza Blade para la definición de plantillas en las vistas. Esta librería permite realizar todo tipo de operaciones con los datos, además de la sustitución de secciones de las plantillas por otro contenido, herencia entre plantillas, definición de layouts o plantillas base, etc.

Los ficheros de vistas que utilizan el sistema de plantillas Blade tienen que tener la extensión .blade.php. Esta extensión tampoco se tendrá que incluir a la hora de referenciar una vista desde el fichero de rutas o desde un controlador. Es decir, utilizaremos view('home') tanto si el fichero se llama home.php como home.blade.php.

En general el código que incluye Blade en una vista empezará por los símbolos @ o {{, el cual posteriormente será procesado y preparado para mostrarse por pantalla. Blade no añade sobrecarga de procesamiento, ya que todas las vistas son preprocesadas y cacheadas, por el contrario nos brinda utilidades que nos ayudarán en el diseño y modularización de las vistas.

El método más básico que tenemos en Blade es el de mostrar datos, para esto utilizaremos las llaves dobles ({{ }}) y dentro de ellas escribiremos la variable o función con el contenido a mostrar:

```
Hola {{ $name }}.  
La hora actual es {{ time() }}.
```

Mostrar un dato solo si existe

Para comprobar que una variable existe o tiene un determinado valor podemos utilizar el operador ternario de la forma:

```
{{ isset($name) ? $name : 'Valor por defecto' }}
```

O simplemente usar la notación que incluye *Blade* para este fin:

```
{{ $name or 'Valor por defecto' }}
```

Comentarios

Para escribir comentarios en Blade se utilizan los símbolos {{-- y --}}, por ejemplo:

```
{{-- Este comentario no se mostrará en HTML --}}
```

Estructuras de control

Blade nos permite utilizar la estructura if de las siguientes formas:

```
@if( count($users) === 1 )  
    Solo hay un usuario!  
@elseif (count($users) > 1)  
    Hay muchos usuarios!  
@else  
    No hay ningún usuario!  
@endif
```

En los siguientes ejemplos se puede ver cómo realizar bucles tipo for, while o foreach:

```
@for ($i = 0; $i < 10; $i++)  
    El valor actual es {{ $i }}  
@endfor  
  
@while (true)  
    <p>Soy un bucle while infinito!</p>  
@endwhile
```



```
@foreach ($users as $user)
    <p>Usuario {{ $user->name }} con identificador: {{ $user->id}}</p>
@endforeach
```

Esta son las estructuras de control más utilizadas. Además de estas *Blade* define algunas más que podemos ver directamente en su documentación [Blade Templates - Laravel - The PHP Framework For Web Artisans](#)

Incluir una plantilla dentro de otra plantilla

En Blade podemos indicar que se incluya una plantilla dentro de otra plantilla, para esto disponemos de la instrucción `@include`:

```
@include('view_name')
```

Además podemos pasarle un array de datos a la vista a cargar usando el segundo parámetro del método `include`:

```
@include('view_name', array('some'=>'data'))
```

Esta opción es muy útil para crear vistas que sean reutilizables o para separar el contenido de una vista en varios ficheros.

Layouts

Blade también nos permite la definición de layouts para crear una estructura HTML base con secciones que serán rellenadas por otras plantillas o vistas hijas. Por ejemplo, podemos crear un layout con el contenido principal o común de nuestra web (head, body, etc.) y definir una serie de secciones que serán rellenados por otras plantillas para completar el código. Este layout puede ser utilizado para todas las pantallas de nuestro sitio web, lo que nos permite que en el resto de plantillas no tengamos que repetir todo este código.

A continuación se incluye un ejemplo de una plantilla tipo layout almacenada en el fichero `resources/views/layouts/master.blade.php`:

```
<html>
<head>
    <title>Mi Web</title>
</head>
<body>
    @section('menu')
        Contenido del menu
    @show

    <div class="container">
        @yield('content')
    </div>
</body>
</html>
```

Posteriormente, en otra plantilla o vista, podemos indicar que extienda el layout que hemos creado (con `@extends('layouts.master')`) y que complete las dos secciones de contenido que habíamos definido en el mismo:

```
@extends('layouts.master')
@section('menu')
    @parent
    <p>Este contenido es añadido al menú principal.</p>
@endsection

@section('content')
    <p>Este apartado aparecerá en la sección "content".</p>
@endsection
```

Como se puede ver, las vistas que extienden un layout simplemente tienen que sobrescribir las secciones del layout. La directiva `@section` permite ir añadiendo contenido en las plantillas hijas, mientras que `@yield` será sustituido por el contenido que se indique. El método `@parent` carga en la posición indicada el contenido definido por el padre para dicha sección.

El método `@yield` también permite establecer un contenido por defecto mediante su segundo parámetro:

```
@yield('section', 'Contenido por defecto')
```